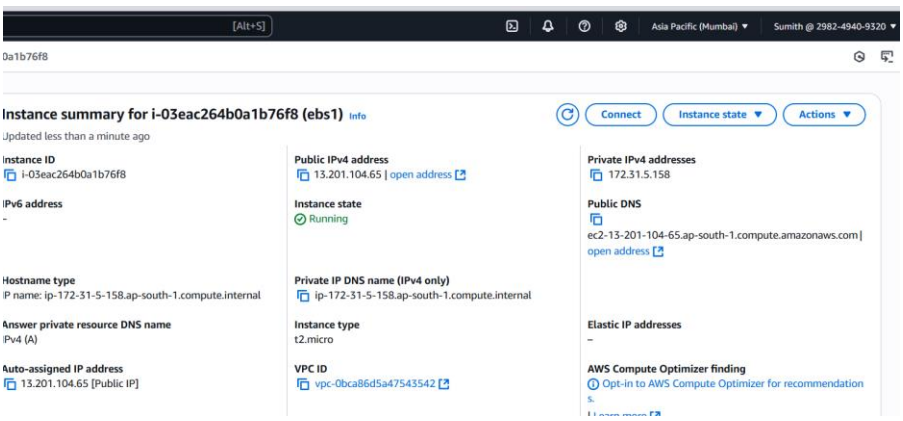
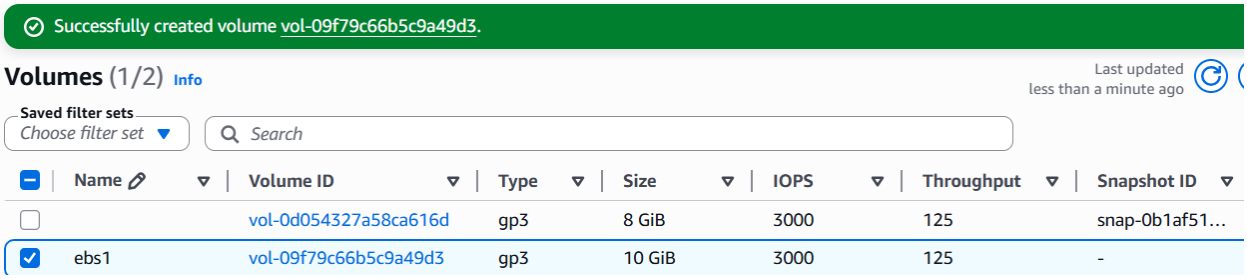
EBS & EFS Practical Exercise

* Task1:
* Create an EBS volume, attach it to an EC2 instance, take a snapshot, and try attaching it to another instance.
* Experiment with EFS by mounting it across multiple EC2 instances and test file sharing.

Step 1: Create an EBS Volume

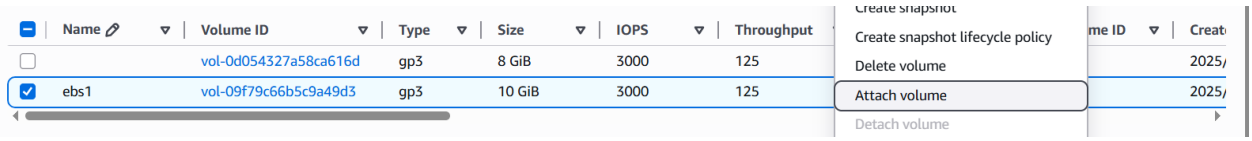
1. Go to the **EC2 Dashboard** > **Elastic Block Store** > **Volumes**.
2. Click **Create Volume**.
   - Select **Volume Type** (e.g., gp2).
   - Choose **Size** (e.g., 8 GiB).
   - **Availability Zone** must match your target EC2 instance.
3. Click **Create Volume**.





Step 2: Attach EBS Volume to an EC2 Instance

1. Select the created volume > **Actions** > **Attach Volume**.
2. Select your **running EC2 instance**.
3. Provide a **device name** (e.g., /dev/xvdf), then **Attach**.



Step 3: Format & Mount EBS Volume on EC2

1. SSH into your EC2 instance.
2. Run:

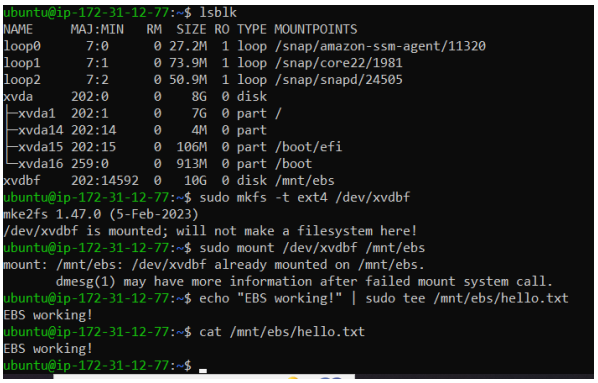
   lsblk # Check if /dev/xvdf appears

   sudo mkfs -t ext4 /dev/xvdf

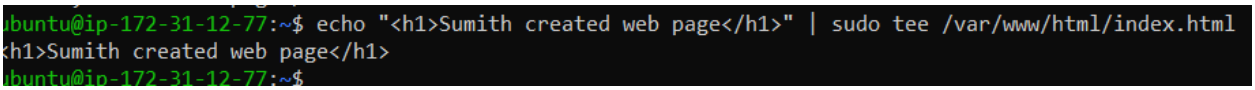
   sudo mkdir /mnt/ebs

   sudo mount /dev/xvdf /mnt/ebs
3. Create a sample file to verify:

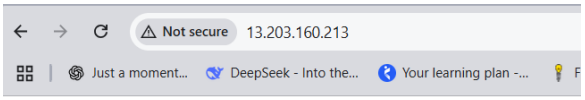
   echo "Hello from EBS" | sudo tee /mnt/ebs/ebs-test.txt



```
ubuntu@ip-172-31-12-77:~$ lsblk
NAME     MAJ:MIN   RM  SIZE RO TYPE MOUNTPOINTS
loop0      7:0      0 27.2M  1 loop /snap/amazon-ssm-agent/11320
loop1      7:1      0 73.9M  1 loop /snap/core22/1981
loop2      7:2      0 50.9M  1 loop /snap/snapd/24505
xvda     202:0      0    8G  0 disk
├─xvda1  202:1      0    7G  0 part /
├─xvda14 202:14     0    4M  0 part
├─xvda15 202:15     0  106M  0 part /boot/efi
└─xvda16 259:0      0  913M  0 part /boot
xvdbf    202:14592  0   10G  0 disk /mnt/ebs
ubuntu@ip-172-31-12-77:~$ sudo mkfs -t ext4 /dev/xvdbf
mke2fs 1.47.0 (5-Feb-2023)
/dev/xvdbf is mounted; will not make a filesystem here!
ubuntu@ip-172-31-12-77:~$ sudo mount /dev/xvdbf /mnt/ebs
mount: /mnt/ebs: /dev/xvdbf already mounted on /mnt/ebs.
       dmesg(1) may have more information after failed mount system call.
ubuntu@ip-172-31-12-77:~$ echo "EBS working!" | sudo tee /mnt/ebs/hello.txt
EBS working!
ubuntu@ip-172-31-12-77:~$ cat /mnt/ebs/hello.txt
EBS working!
ubuntu@ip-172-31-12-77:~$
```



```
ubuntu@ip-172-31-12-77:~$ echo "<h1>Sumith created web page</h1>" | sudo tee /var/www/html/index.html
<h1>Sumith created web page</h1>
ubuntu@ip-172-31-12-77:~$
```



Sumith created web page

Name	Volume ID	Type	Size	IOPS	Throughput	Snapshot ID	Source volume ID
	vol-0522dce30d7b7d6c5	gp3	8 GiB	3000	125	snap-0b1af51...	-
ebs2	vol-0a280f28762c7cc0e	gp3	10 GiB	3000	125	snap-0083723...	-
ebs1	vol-09f79c66b5c9a49d3	gp3	10 GiB	3000	125	-	-

ebs2	i-066fb20eacdfcc48f	Running	t2.micro	Initializing	View alarms +	ap-south-1b	ec2-13-2:
ebs1	i-03eac264b0a1b76f8	Terminated	t2.micro	–	View alarms +	ap-south-1b	–
ebs	i-05f9e05bb8e9587fc	Running	t2.micro	2/2 checks passec	View alarms +	ap-south-1b	ec2-13-2(

Saved filter sets
Choose filter set
Search
1

Name	Volume ID	Type	Size	IOPS	Throughput	Snapshot ID	Source volume ID
	vol-0522dce30d7b7d6c5	gp3	8 GiB	3000	125	snap-0b1af51...	-
ebs2	vol-0a280f28762c7cc0e	gp3	10 GiB	3000	125	snap-0083723...	-
	vol-0ab8584185640cb75	gp3	8 GiB	3000	125	snap-0b1af51...	-
ebs1	vol-09f79c66b5c9a49d3	gp3	10 GiB	3000	125	-	-

Step 5: Create a New Volume from Snapshot & Attach to Another EC2

1. Go to **Snapshots** > select your snapshot > **Create Volume**.
2. Make sure to use the **same AZ** as your second EC2.
3. Attach it to another EC2 instance > SSH into that instance.
4. Mount it:

bash

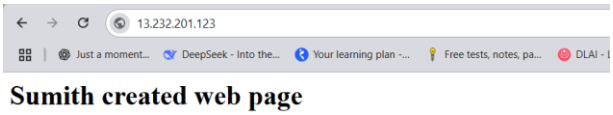
CopyEdit

lsblk

sudo mkdir /mnt/ebs2

sudo mount /dev/xvdf /mnt/ebs2

cat /mnt/ebs2/ebs-test.txt



Sumith created web page

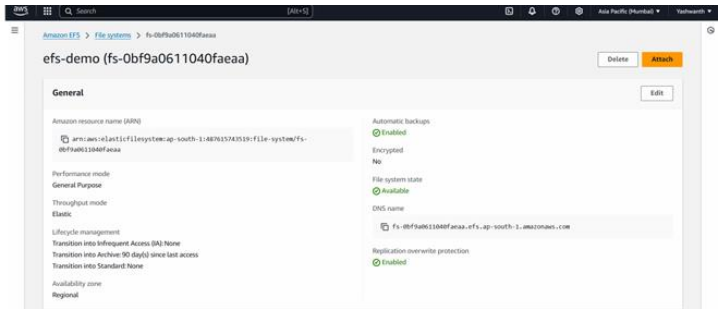
EFS PRACTICAL TASK

◈ Step 1: Create an EFS File System

1. Go to **EFS** dashboard > **Create file system**.
2. Choose **VPC**, leave default settings.
3. Click **Next** until **Create File System**.

Step 2: Mount EFS to Multiple EC2 Instances

1. Ensure EC2 instances are in the **same VPC & security group allows NFS (port 2049)**.
2. SSH into **each EC2 instance**.
3. Install NFS client:




```
sudo yum install -y nfs-utils     # For Amazon Linux
sudo apt install -y nfs-common   # For Ubuntu
```

1. Mount EFS:

   ```
   sudo mkdir /mnt/efs
   sudo mount -t nfs4 -o nfsvers=4.1 [Your_EFS_DNS]:/ /mnt/efs
   ```

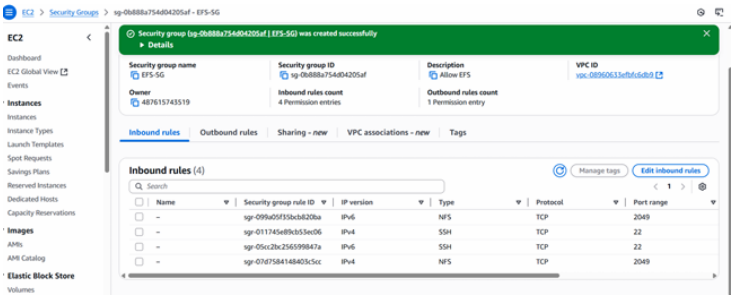
   Replace [Your_EFS_DNS] with DNS from EFS console (e.g., fs-xxxx.efs.us-east-1.amazonaws.com).

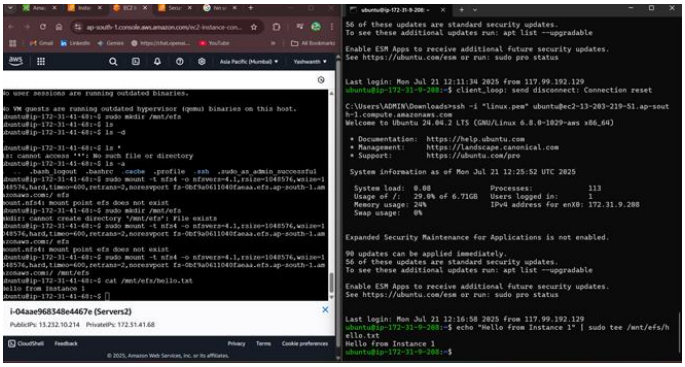
2. Test file sharing:
   - On Instance 1

     ```
     echo "Shared via EFS" | sudo tee /mnt/efs/test.txt
     ```

   - On Instance 2:

     ```
     cat /mnt/efs/test.txt     # Should display "Shared via EFS"
     ```





Task: Auto Scaling & Load Balancer Setup

🛠 1. Create a Launch Template from Existing EC2

This template will be used to auto-launch instances when needed.

- Go to **EC2 Dashboard > Launch Templates > Create launch template**
- Name it: e.g., web-app-template
- Select:
  - **AMI**: same as your running EC2
  - **Instance Type**: same (e.g., t2.micro)
  - **Key pair**, **Security group**, **User data** (optional)
- Click **Create launch template**

✅ You now have a reusable template for Auto Scaling.

🔁 2. Create Auto Scaling Group (ASG)

- Go to **EC2 > Auto Scaling Groups > Create**
- **Choose Launch Template** → Select the one you just created
- **VPC & Subnet**: Choose same as EC2
- **Load Balancer (optional but recommended)**:

    - Select an **existing Target Group** or create a new **Application Load Balancer (ALB)**
- **Set capacity**:
    - Minimum: 1
    - Desired: 1
    - Maximum: 2 or more (as you wish)
- Health checks: Enable **EC2 or ELB** based
- Tag: Name = AutoEC2

✅ ASG now manages EC2 lifecycle automatically.

💣 3. Test Auto-Replacement by Termination

- Go to **EC2 > Instances**
- Find the EC2 launched by ASG (check tag or instance ID)
- **Terminate it manually**

🎯 **Expected behavior**:

- Auto Scaling Group detects missing instance
- Launches a **new one automatically**
- Within a few minutes, the new instance appears with **same launch template settings**

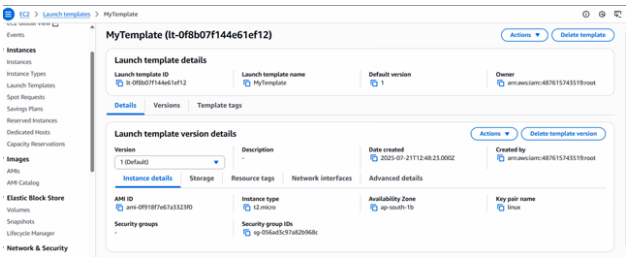
▫️ Optional: Connect with Load Balancer

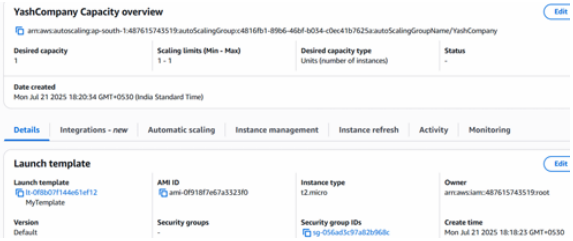
If using an **Application Load Balancer (ALB)**:

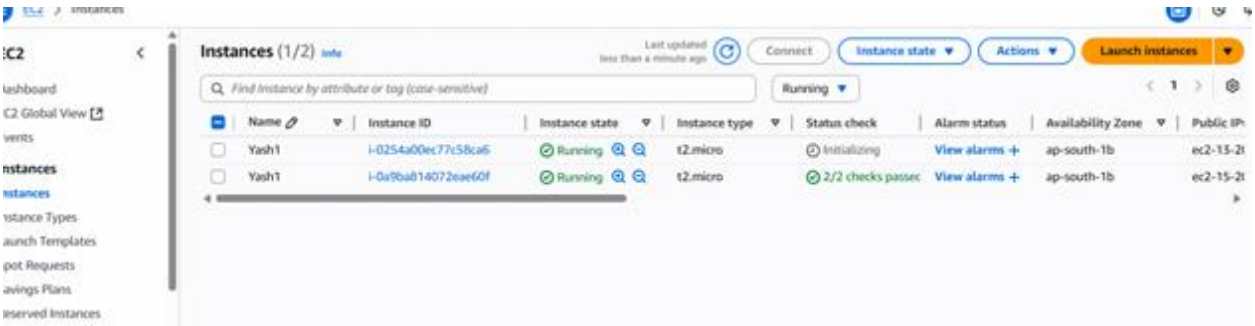
- Go to **EC2 > Load Balancers**
- Copy the **DNS name** of the ALB
- Paste in browser: should serve from backend EC2 managed by ASG

✅ This proves:

- Auto Scaling is working
- Load Balancer is routing requests to available instances







DOCUMENTS

EBS vs. EFS vs. S3 – AWS Storage Comparison:

Amazon EBS (Elastic Block Store)

📌 **Description:**

EBS is **block-level storage** designed to be attached to **a single EC2 instance** at a time, like a virtual hard disk. It acts as persistent storage for operating systems, databases, and applications that require fast, low-latency access to data.

✅ **Use Cases:**

- Boot volumes for EC2
- Databases like MySQL, PostgreSQL
- Applications requiring transactional I/O (e.g., ERP)
- Custom file systems

⚙️ **Key Features:**

- Comes in various volume types: gp3, io2, sc1, st1
- Supports snapshots (backed by S3) for backups
- Data remains even after EC2 shutdown
- Encryption using AWS KMS

💰 **Pricing:**

- Charged per GB/month (e.g., gp3 ~$0.08/GB)
- Additional costs for provisioned IOPS (for io2)
- Snapshots: ~$0.05/GB/month
- More cost-effective than EFS for single-instance workloads

🟤 **Amazon EFS (Elastic File System)**

📌 **Description:**

EFS provides **shared, scalable file storage** that can be mounted concurrently by multiple EC2 instances across Availability Zones. It uses the **NFS protocol** and behaves like a managed NAS (Network Attached Storage).

✅ **Use Cases:**

- Shared storage for web servers and containers
- Machine learning pipelines
- Development environments
- CMS like WordPress (shared asset folders)

⚙️ **Key Features:**

- Automatically grows and shrinks as files are added or deleted
- Supports two performance modes (General Purpose, Max I/O)
- Supports lifecycle management (automatically moves files to infrequent access tier)
- Offers regional redundancy (multi-AZ)

💰 **Pricing:**

- Standard: ~$0.30/GB/month
- Infrequent Access tier: ~$0.025/GB/month + per-access fee
- Much costlier than EBS if frequently accessed
- Better suited for workloads that truly require shared access

---

□ Amazon S3 (Simple Storage Service)

📌 **Description:**

S3 is **object storage** built for the cloud, capable of storing any amount of unstructured data. Unlike EBS or EFS, S3 does not support traditional file system mounting — instead, data is accessed via APIs, SDKs, or S3-compatible tools.

✅ **Use Cases:**

- Static website hosting
- Big data analytics (data lake)
- Media storage (images, videos)
- Backup and archival (via Glacier)

⚙️ **Key Features:**

- 11 9's durability (99.999999999%)
- Supports versioning, encryption, access control policies
- Lifecycle rules to transition data across storage tiers (Standard → Infrequent → Glacier)
- Accessed over HTTP/HTTPS using RESTful API

💰 **Pricing:**

- S3 Standard: ~$0.023/GB/month
- S3 IA: ~$0.0125/GB/month
- Glacier: ~$0.004/GB/month
- Additional charges for PUT/GET requests and data transfer

Summary: When to Use What?

Need	Best Option
Attach persistent disk to EC2	**EBS**
Share files across EC2 instances	**EFS**
Store backups, images, static files	**S3**
Require low-latency transactional I/O	**EBS**
Need scalable shared storage	**EFS**
Need web-scale, durable object storage	**S3**

💡 **Final Thought:**

- **EBS** is like your laptop's SSD: fast, private, and attached.
- **EFS** is like a shared network drive: good for collaboration.
- **S3** is like a cloud hard drive: scalable, durable, and globally accessible

Vertical vs. Horizontal Scaling: Real-World Insights & Comparison

🌐 Overview

In cloud computing and system architecture, **scaling** is the process of adjusting resources to meet demand. There are two primary methods:

- **Vertical Scaling (Scale Up)**: Adding more power (CPU, RAM, SSD) to an existing machine.
- **Horizontal Scaling (Scale Out)**: Adding more machines/nodes to handle the load in parallel.

🧱 Real-World Examples

Company	Scaling Type	How They Use It
Netflix	**Horizontal**	Distributes traffic across thousands of EC2 instances using auto-scaling groups
Facebook	**Horizontal**	Shards user data across servers; uses Memcached clusters for distributed caching
Amazon	**Horizontal**	Leverages microservices architecture with containers (ECS, EKS)
LinkedIn	**Combination**	Uses vertical scaling for Kafka brokers and horizontal for web servers
Small Startups	**Vertical**	Start with vertically scaled VMs for simplicity and cost efficiency

---

⚖️ Pros & Cons

▲ Vertical Scaling (Scale-Up)

Increase capacity of a **single** machine.

✅ **Pros:**

- Simpler to implement (especially on monolithic apps)
- No changes in application logic
- Lower initial complexity
- Easier to manage in small deployments

❌ **Cons:**

- Limited by hardware limits (CPU/RAM ceiling)
- Downtime may be required during upgrades
- Poor fault tolerance — if one machine fails, service is down
- Not cost-effective beyond a point

🔁 Horizontal Scaling (Scale-Out)

Increase capacity by adding **multiple machines**.

✅ **Pros:**

- High availability and fault tolerance
- Infinite scalability (in theory)
- Ideal for cloud-native, distributed systems
- Load balancing can distribute traffic efficiently

❌ **Cons:**

- More complex infrastructure (e.g., load balancers, data sharding)
- Requires stateless app design or session management
- Data consistency across nodes can be challenging
- Increased ops cost in managing multiple nodes

🔍 Choosing the Right Scaling Method

a	Vertical Scaling	Horizontal Scaling
pps/startups	nple and cost-effective	necessary overhead
cale web apps	rdware limits	les elastically with demand
monoliths	ier to adapt	y require major refactor
native microservices	s flexibility	signed to scale horizontally
constraints	eaper to begin with	stlier at first but pays off with scale

---

🧠 Summary Analogy

- **Vertical Scaling** = Upgrading your car engine — more power, same vehicle.
- **Horizontal Scaling** = Adding more cars to a fleet — better throughput, more complexity.

📌 Final Thought

- Most modern, large-scale applications (like Netflix, Amazon, and Facebook) **rely heavily on horizontal scaling** for resilience and scalability.

- **Vertical scaling** is suitable for quick performance boosts or simpler systems — but has physical and risk limitations.

Troubleshooting AWS Auto Scaling Group (ASG) Misconfiguration

□ Scenario: Intentional Misconfiguration

Problem Setup:

We create an Auto Scaling Group (ASG) using a **non-functional Launch Template** — either referencing a missing AMI ID or incorrect key pair name.

⚙ **Configuration Details**

Component	Value
Launch Template	LT-Broken Template
AMI ID	ami-0xyz123wrongid (nonexistent)
Instance Type	t2.micro
Key Pair	Non Existent Key Pair
ASG Min/Max/Des.	1/3/1
Target Group	Attached (ELB)

❗ **Error Messages Observed**

1. EC2 Launch Failure

In the **EC2 Auto Scaling > Activity History**, you might see:

"Failed to launch instance. Reason: The image id '[ami-0xyz123wrongid]' does not exist"

🛠 **Fix**:

- Go to EC2 → AMIs → Check for a valid AMI ID.
- Update the Launch Template version with a correct AMI (e.g., Amazon Linux 2).

---

2. Key Pair Not Found

"The key pair 'Non Existent Key Pair' does not exist"

🛠 **Fix**:

- Go to EC2 → Key Pairs → Create or choose an existing one.
- Edit Launch Template and associate the correct key.

---

3. Health Check Failures in ELB

"Instance failed ELB health checks and was replaced"

🛠 **Fix**:

- Check if the launched instance has the proper web server installed (Apache, Nginx, etc.).
- Verify security group allows HTTP/HTTPS (port 80/443).
- Ensure the instance's user data properly installs the service.

---

4. Insufficient Capacity or AZ Mismatch

"Capacity error or AZ mismatch"

🛠 **Fix**:

- Make sure your ASG's subnets span across **available AZs** in the selected region.
- Ensure instance type is available in those AZs (especially with spot instances).

□ Steps to Resolve

1. **Audit the Launch Template**
   - Verify AMI, instance type, key pair, user data script.
2. **Update Launch Template**
   - Create a **new version** with fixed values — do **not** edit existing version.
3. **Update ASG to use correct template version**
   - Set default version or manually select a version in ASG config.
4. **Manually trigger instance refresh**
   - Force instance replacement using "Instance refresh" if needed.
5. **Review Scaling Policies**
   - Ensure target tracking or step scaling policies are not firing too quickly.

✅ **Post-Fix Behavior**

Once properly configured:

- **New instances launch successfully**
- **ELB shows healthy status**

- **Scaling in/out events work as expected**
- **CloudWatch alarms are stable**

□ Tips

Issue	Resolution
Wrong AMI or key	Fix in Launch Template → create new version
Launch config vs. template mismatch	Ensure ASG uses Launch Template **OR** Launch Config, not both
Unhealthy after launch	Fix user data, security groups, and ELB health checks
Stuck at 0 instances	Check subnet, AZ, quota, and desired capacity settings

---

Build AWS Lambda Triggered by S3 Uploads:

◈ Use Case:

Whenever a file is uploaded to an S3 bucket, a Lambda function will be triggered to log the filename and metadata.

🔧Step-by-Step Guide:

1. Create an S3 Bucket

- Go to AWS S3 → Create bucket (e.g., lambda-trigger-bucket).
- Enable versioning (optional) and allow public/private based on use case.

2. Create IAM Role for Lambda

- Go to IAM → Roles → Create Role.
- Select **Lambda** as use case.
- Attach policies:
  - AmazonS3ReadOnlyAccess
  - CloudWatchLogsFullAccess
- Name: LambdaS3ExecutionRole.

3. Create the Lambda Function

- Go to Lambda → Create function.
- Name: S3TriggerLogger
- Runtime: Python 3.12

- Execution role: Use existing (LambdaS3ExecutionRole)

4. Lambda Code (Python)

```
import json
import logging

logger = logging.getLogger()
logger.setLevel(logging.INFO)

def lambda_handler(event, context):
    for record in event['Records']:
        bucket = record['s3']['bucket']['name']
        key = record['s3']['object']['key']
        logger.info(f"New file uploaded: s3://{bucket}/{key}")

    return {
        'statusCode': 200,
        'body': json.dumps('Success!')
    }
```

5. Add S3 Trigger to Lambda

- In Lambda → Triggers → Add trigger.
- Select **S3**, choose your bucket.
- Event type: PUT (i.e., on upload)
- Enable trigger.

✅ PART 2: 📊 Cost Monitoring with AWS Cost Explorer

🎯 **Goal:**

Track how much you're spending on Lambda, S3, and related services during experiments.

Steps:

1. **Enable Cost Explorer**
   - Go to Billing → Cost Explorer → Enable (if not already).
   - Takes ~24 hours to populate data.
2. **Set Up a Daily Cost Report**
   - Choose **Daily granularity**, group by **Service**.

- Filter for services: Lambda, S3, CloudWatch.
- Set alerts using **AWS Budgets**.

3. **Pro Tip**: Use **Tags**
   - Tag your resources (Project: LambdaS3Demo) and filter in Cost Explorer by tag.

✅ PART 3: 🗀 Upload Scripts to GitHub Portfolio

```
aws-lambda-s3-trigger/
├── lambda_function.py          # Lambda handler code
├── create_lambda.py            # Boto3 script to automate deployment
├── s3_upload_trigger_test.py   # Script to upload test files
├── README.md                   # Project overview
└── requirements.txt            # Boto3, etc.
```

□ create_lambda.py (Boto3 deployment script)

```
import boto3

client = boto3.client('lambda')

with open('lambda_function.zip', 'rb') as f:
    zipped_code = f.read()

response = client.create_function(
    FunctionName='S3TriggerLogger',
    Runtime='python3.12',
    Role='arn:aws:iam::<your-account-id>:role/LambdaS3ExecutionRole',
    Handler='lambda_function.lambda_handler',
    Code=dict(ZipFile=zipped_code),
    Timeout=15,
    MemorySize=128,
    Publish=True,
)
print("Lambda Created:", response['FunctionArn'])
```

🔧 **Automate the Build (Optional)**

- Use GitHub Actions to deploy Lambda with AWS CLI.
- Add a workflow file in .github/workflows/deploy.yml.